AI-Native Engineer Capstone

TelcoLane — Subscriber & Plan Lifecycle Management Platform

Business Case BC-AINE-020 · Domain: Telecom — Subscriber Operations

1. Project Identity

Business Case Title	Subscriber & Plan Lifecycle Management Platform
Business Case ID	BC-AINE-020
Domain	Telecom — Subscriber Operations
Project Type	AI-Native Engineering Capstone
Cohort	(filled by Operations team)

2. Engagement Overview

Duration	20 hours
Format	Individual
Evaluation Mode	Automated review of submitted Git repository against the AI-Native Engineering Rubric (21 parameters / 100 marks across 5 categories)
Submission	Push final code to your assigned Virtusa GitLab repository by the cohort cut-off
Review Output	Per-learner Excel report (Summary, Categories, Scorecard, Detailed, Improvement)
Grade Bands	Distinction ≥ 90 · Merit 75–89 · Pass 60–74 · Not Yet Passed < 60

What is evaluated: AI-native engineering practices (specs, agents, skills, hooks, CLAUDE.md hierarchy, plugin packaging, MCP, Harness plugin usage), business understanding (context, rules, AC), architectural discipline (structural rules, TDD), technical implementation (code, tests, coverage, UI validation), and CI/CD integration (PR-driven workflow, Claude in CI).

What is not evaluated: the specific technology stack chosen, the visual polish of the UI, or the use of any AI provider beyond Claude Code.

3. Problem Statement & Expected Solution

3.1 Problem

A telecom operator wants to consolidate subscriber and plan management into a single platform — handling activations, plan upgrades and downgrades, suspensions, pro-rata billing, and port-out workflows — under one constraint: no manual coding. All external integrations (network provisioning,

MNP backend, payment gateway) are stubbed; every line of production code must be generated by Claude Code agents under your supervision.

3.2 Your Role

AI-Native Engineer — architect, supervisor, and quality gatekeeper. You write specifications, configure agents, install guardrails, and review pull requests. You do not write production code by hand.

3.3 Expected Solution

A working platform delivered as a Git repository that demonstrates:

- A complete AI-native engineering substrate built on the Claude Harness Engine plugin foundation, extended with telecom-subscriber-operations-specific specs, agents, skills, hooks, commands, and MCP integration.
- A functional telecom subscriber lifecycle platform built end-to-end by Claude Code agents — covering subscriber activation, plan catalog management, plan-change workflow with pro-rata billing, suspension and reactivation, port-out cooling-period workflow, CSR overrides, and admin reporting — with no hand-written production code.
- An evidence trail (PRs, agent logs, post-mortems, sprint contracts) proving the work was agent-led.

3.4 Applicable Rules

- **No-Hand-Coding Rule.** All production code, tests, and migrations must be generated by Claude Code agents. Hand-edits are limited to specs, CLAUDE.md, agent definitions, hooks, skills, and other substrate files.
- **Spec-Is-Truth Rule.** Whenever a spec and code disagree, the spec wins. Update the spec deliberately and regenerate the code.
- **PR-Only Merge Rule.** No direct commits to main. All changes flow through agent-reviewed pull requests.
- **Synthetic-Data Rule.** Use only dummy data. Do not use Virtusa confidential or customer-sensitive data for this project.

4. Technology Stack

Choose any stack that matches your background. The stack does not affect evaluation.

Layer	Options
Frontend	React · Angular · Vue · server-rendered (Thymeleaf / Razor)
Backend	Java + Spring Boot · .NET + ASP.NET Core Web API · Python + FastAPI · Node.js + Express / NestJS
Database	In-memory (H2 / SQLite) · RDBMS (MySQL / MariaDB / SQL Server) · NoSQL (MongoDB / DynamoDB)
Build & Dependency Management	Maven / Gradle · dotnet CLI · pip + Poetry · npm / pnpm

Layer	Options
Testing	JUnit / NUnit / pytest / Vitest · Playwright (E2E) · ArchUnit / dependency-cruiser / import-linter
CI	GitLab CI (preferred for Virtusa) · GitHub Actions, with Claude Code Action integrated
MCP Server	Playwright MCP
Required Plugin	Claude Harness Engine plugin (Generator–Evaluator harness for autonomous long-running development)

5. Acceptance Criteria & Non-Functional Requirements

5.1 Functional Acceptance Criteria

ID	Criterion
AC-01	Subscriber can self-register with mobile number, identity proof reference (stubbed), and preferred plan type (PREPAID / POSTPAID); the subscription is created in PENDING_KYC state
AC-02	Subscriber lifecycle states PENDING_KYC → ACTIVE → SUSPENDED / TERMINATED / PORT_OUT_REQUESTED → PORTED_OUT are enforced; invalid transitions raise InvalidSubscriberStateException
AC-03	Plan catalog supports both prepaid (validity-based) and postpaid (billing-cycle-based) plans with versioned rules; published plan versions are immutable
AC-04	Subscriber can upgrade or downgrade a plan; the change is effective immediately for prepaid and from the next billing cycle for postpaid; plan-change rules (e.g., minimum tenure) are enforced
AC-05	Pro-rata billing on mid-cycle postpaid plan changes: charges_total = (days_used / cycle_days × old_plan_rent) + (days_remaining / cycle_days × new_plan_rent); the invariant charges_total ≥ 0 must hold
AC-06	Suspension: a subscriber moves to SUSPENDED state on request or non-payment; consumption is halted (stubbed); reactivation restores the prior state and resumes consumption
AC-07	Port-out request initiates a 7-day cooling period; subscriber state moves to PORT_OUT_REQUESTED; the request can be cancelled within the cooling period and the subscriber returns to ACTIVE
AC-08	Activation rules: KYC must be VERIFIED, the dealer code must be valid, and the MNP validation (stubbed) must complete; failure on any rule raises a specific exception with a reason code

ID	Criterion
AC-09	Customer Service Representative (CSR) can override an activation or plan-change rejection with a reason code; the override is audited with actor and timestamp
AC-10	Admin can view activation funnel (registered → active), monthly churn rate, plan-mix distribution, and stubbed Average Revenue Per User (ARPU) trends

Note. *Each AC must have at least one test referencing its AC-NN identifier.*

5.2 Non-Functional Requirements

ID	Requirement
NFR-01	Pro-rata charges, plan rents, and usage charges are computed in fixed-point (BigDecimal / decimal) — never floating-point
NFR-02	Plan versions, subscriber state transitions, billing records, port-out events, and CSR overrides are append-only
NFR-03	Subscriber PII (Aadhaar reference, PAN reference, full mobile number) is masked in logs and never persisted in plaintext where avoidable
NFR-04	Authentication boundary at controller layer; subscriber / CSR / admin / dealer roles are separated; a subscriber can only see their own subscription
NFR-05	Database migrations and plan-catalog migrations are append-only
NFR-06	Structured JSON logs with request correlation IDs and masked subscriber_id
NFR-07	Health endpoint returns 200 within 1 second of a successful startup
NFR-08	Architecture rules enforced as tests — published plans cannot be mutated; a subscriber cannot hold two ACTIVE subscriptions for the same mobile number; the pro-rata invariant must hold for any mid-cycle plan change

6. Functional Scope

6.1 Customer / End-User Functionality (User Journeys)

- **Register & Activate.** Self-register with mobile number, identity proof reference, and preferred plan type; complete activation through stubbed KYC and MNP checks.
- **View Plan & Usage.** View current plan, validity / billing-cycle, and consumption (data, voice, SMS — stubbed counters).
- **Change Plan.** Upgrade or downgrade plan; preview the pro-rata charge breakdown before confirming.
- **Suspend / Resume.** Suspend own subscription temporarily; reactivate when ready.
- **Port Out.** Initiate a port-out request; cancel within the 7-day cooling period if needed.
- **Billing / Recharge.** View postpaid bill or top-up prepaid balance (stubbed payment).

6.2 Internal & System Functionality

- **Plan Catalog Admin.** Maintain prepaid and postpaid plans with versioned rules (rent, validity, data quota, voice minutes, SMS, applicable tenure).
- **Activation Engine.** Enforce activation rules (KYC, dealer, MNP) and route to CSR queue when an exception is raised.
- **Plan-Change Engine.** Compute pro-rata charges for postpaid plan changes; apply prepaid changes immediately.
- **Port-Out Workflow.** Manage the 7-day cooling-period state machine; allow cancellation; finalise port-out on completion.
- **CSR Workbench.** Handle complaints, activation rejections, plan-change exceptions, and override audits.
- **Reports & Dashboards.** Activation funnel, churn rate by reason, plan-mix distribution, stubbed ARPU trends.

6.3 Out of Scope

You do not need to build:

- Real network / HSS / OCS provisioning — provisioning calls are stubbed
- Real KYC document verification (Aadhaar e-KYC, video KYC) — flag-based stub only
- Real MNP backend integration — workflow states only, no real network operator handshake
- Real payment gateway integration (recharge / bill payment) — stubbed responses
- 5G slicing, IoT / M2M plans, enterprise plans, and bulk SIM activation
- Number-inventory management (assume the mobile number is provided and reserved upstream)
- Production deployment, secret management, multi-region

7. AI-Native Implementation Expectations

Foundation. Install the **Claude Harness Engine plugin** first. The harness ships 7 base agents (planner, generator, evaluator, security-reviewer, test-engineer, design-critic, ui-designer), 12+ hooks, base commands, sprint contracts, an evaluator wired to Playwright MCP, and a self-healing loop. On top of that foundation, your repository must add the project-specific substrate listed in this section. The rubric scores the final repository — both Harness-supplied and learner-added artifacts contribute, but the thresholds below are calibrated so that learners who only install Harness without customizing for the project domain will not meet them.

7.1 Claude Code Specs (40 marks)

- specs/app_spec.md (root spec) and specs/<feature>_spec.md per major feature, each with explicit Acceptance Criteria sections
- Layered CLAUDE.md — root + per-module + per-folder; AGENTS.md as a table of contents only
- At least 3 project-specific skills under .claude/skills/ added by you (examples: plan-rule-evaluator, pro-rata-calculator, subscriber-state-validator, port-out-cooling-checker, kyc-validation-stub; or technical: state-machine-validator, archtest-author, audit-tracer)
- At least 2 project-specific custom commands under .claude/commands/ added by you (beyond Harness defaults)

- At least 2 project-specific hooks under .claude/hooks/ added by you (examples: plan-immutability-check.sh, pii-masking-check.sh, pro-rata-invariant-check.sh)
- At least 3 project-specific agents under .claude/agents/ added by you. These may be **domain agents** (e.g., activation-agent, plan-change-agent, port-out-agent, billing-agent, csr-override-agent), **technical agents** that strengthen the engineering substrate (e.g., state-machine-validator-agent, archtest-author-agent, audit-tracer-agent), or a mix of both. You may rename Harness's generator → implementer, evaluator → reviewer, test-engineer → tester to align with canonical naming
- At least one programmatic Claude Agent SDK usage in scripts/
- Your own plugin.json packaging your project's substrate (in addition to Harness's plugin.json)
- At least one project-specific debugging-log or post-mortem document under docs/ evidencing environment-first resolution
- Playwright MCP configured in .mcp.json and actively used by the evaluator

7.2 Business Understanding (20 marks)

- docs/business-case.md covering: problem, target users, success metrics, domain rules, value proposition. This is the document the reviewer plugin reads — see Section 8
- At least 4 rule / policy / validator files under src/domain/ (or equivalent) implementing telecom-subscriber-operations business rules
- Frontend evidence (any of: React / Angular / Vue) with at least one responsive layout
- Acceptance criteria expressed in testable form — Given-When-Then or AC-NN identifiers — for at least 4 specs

7.3 Architectural Discipline (8 marks)

- Architecture-test directory (tests/architecture/ or equivalent) with at least 3 structural tests
- ArchUnit / dependency-cruiser / import-linter configured to enforce layering rules
- TDD discipline document at docs/tdd.md and at least 10 test files committed across history showing red–green–refactor pattern

7.4 Technical Implementation (26 marks)

- docs/architecture.md with at least 1 architecture diagram (Mermaid or image) and an explicit layered structure (controllers / services / repositories / domain)
- Substantive codebase with at least 3000 lines of meaningful generated code
- Playwright (or equivalent) for UI validation with at least one snapshot directory
- Tests tagged with AC IDs — every spec AC has at least 1 corresponding test
- At least 20 unit tests with a coverage artefact (coverage.xml, lcov.info, or equivalent) and the coverage tool declared in dependencies

7.5 CI/CD (6 marks)

- .gitlab-ci.yml (or .github/workflows/) with the build and test pipeline
- Claude Code Action (or equivalent) wired into the CI workflow
- At least 3 PR-driven merges (use git merge --no-ff to preserve PR history); zero direct commits to main

8. Expected Outcomes & Deliverables

By the end of 20 hours, the submitted Git repository must contain the Mandatory items below. Good-to-Have items differentiate Merit and Distinction submissions.

8.1 Mandatory

1. **Working Application.** Telecom subscriber lifecycle platform deployable locally via a single command (./mvnw spring-boot:run, npm start, dotnet run, or equivalent) with seed data (plan catalog, sample subscribers, dealer master) and a README quick-start.
2. **docs/business-case.md.** The business context document the reviewer plugin reads. Use this brief's Sections 3 and 6 as the source; rewrite in your own words covering problem, users, success metrics, domain rules, and value proposition.
3. **Specifications.** specs/app_spec.md plus per-feature specs (plan-catalog, activation, plan-change, suspension, port-out, csr-override, reports) with explicit Acceptance Criteria sections.
4. **Layered CLAUDE.md Hierarchy.** Root + per-module + per-folder; AGENTS.md as a table of contents.
5. **Project Agent Substrate.** .claude/agents/, .claude/skills/, .claude/commands/, .claude/hooks/ — Harness foundation plus your project-specific additions as listed in Section 7.1.
6. **Claude Harness Engine Plugin Integration.** Plugin installed; project scaffolded via the harness; at least one full sprint cycle (Generator → Evaluator → ratcheting → PR) driven through it. Sprint contracts and evaluator outputs committed under sprint-contracts/ and specs/reviews/.
7. **Plugin Manifest.** Your own plugin.json at the repository root packaging your telecom-subscriber substrate; .mcp.json with Playwright MCP active.
8. **Test Suite.** Unit tests (incl. pro-rata calculation across boundary dates, state-transition validity, plan-immutability checks, port-out cooling-period scenarios, role-boundary checks), architecture tests, AC-tagged tests, Playwright UI tests, coverage report — all committed.
9. **CI Pipeline.** .gitlab-ci.yml (or equivalent) with Claude Code Action wired in; passing build on main.
10. **PR-Driven Git History.** At least 3 PR-driven merges; zero direct pushes to main.

8.2 Good-to-Have

11. **Architecture & TDD Documentation.** docs/architecture.md with at least one diagram (C4 context or sequence diagram for the activation-to-port-out flow); docs/tdd.md describing the red–green–refactor approach with one worked example (e.g., AC-05 pro-rata boundary-date test).
12. **Autonomous Fix Loop Evidence.** At least one recorded loop where an agent detected a failure (e.g., a pro-rata rounding error at a leap-year boundary, or a suspended subscriber receiving a plan-change notification), reproduced, fixed, validated, and opened a PR. Capture the trace under docs/fix-loops/.
13. **Knowledge Deposit Log.** docs/knowledge-deposits.md recording recurring mistakes encoded back into rules, hooks, or skills (e.g., a hook that blocks any code path computing pro-rata charges without invariant checks).
14. **Multiple Sprint Cycles.** More than one harness-driven sprint covering: plan catalog + activation → plan-change + pro-rata → suspension + port-out → CSR overrides + reports.

15. **Bonus Agents.** Beyond the required additions: janitor, performance-auditor, doc-writer, or other domain-specific or technical agents.

On completion of the capstone, you will have demonstrated the skill the industry is seeking: building a telecom subscriber lifecycle platform with rule-driven activation, accurate pro-rata billing, port-out compliance, and audited state transitions — without writing the code yourself, while maintaining full control of quality, security, and architecture through AI-native engineering practices.